

.NET MICROSERVICES MASTERY

Mikroservis Mimarileri – Altyapı Ön Hazırlık Dökümanı



Neominal Akademi
2026

Mikroservis Mimarilerinde Kullanılan Altyapılar

Doküman Türü: Teorik Ön Hazırlık

İlişkili Eğitim: .NET Microservices Mastery

Kapsam: docker-compose.infra.yml ile ayağa kaldırılan tüm altyapı bileşenlerinin kavramsal temeli

1. Dökümanın Amacı

Bu döküman, mikroservis mimarisine geçişte ihtiyaç duyulan altyapı bileşenlerinin her birinin ne olduğunu, hangi problemi çözdüğünü ve neden gerekli olduğunu teorik düzeyde açıklar. Pratik kurulumu (docker compose up, endpoint testleri vb.) geçmeden önce katılımcının:

- Her bileşenin ait olduğu mimari kategoriye (veri, mesajlaşma, gözlemlenebilirlik, kimlik/güvenlik, service discovery) ayırt edebilmesini,
- Bileşenler arasındaki temel farkları (örn. Kafka vs RabbitMQ, YARP vs Ocelot) gerekçeleriyle açıklayabilmesini,
- Bu bileşenlerin bir isteğin uçtan uca yaşam döngüsünde nasıl bir araya geldiğini görebilmesini

hedefler.

2. Neden Bu Kadar Çok Altyapı Bileşenine İhtiyaç Var?

Monolitik bir uygulamada genelde tek bir veritabanı, tek bir process ve dosya sistemine yazılan basit loglar yeterlidir. Mikroservis mimarisine geçildiğinde ise şu yeni problemler ortaya çıkar ve her biri ayrı bir altyapı bileşeniyle çözülür:

Monolitte kolay olan	Mikroservislerde neden zorlaşır	Çözen bileşen kategorisi
Fonksiyon çağrısıyla veri paylaşımı	Servisler farklı process'lerde, verinin tutarlı şekilde paylaşılması/senkronize edilmesi gerekir	Veri katmanı + Event-Driven mesajlaşma
Tek process içinde asenkron iş kuyruğu	Arka plan işinin, servis çökse bile kaybolmaması, izlenebilir olması gerekir	Background Job Processing
Uygulama içi kullanıcı doğrulama	Onlarca servisin her birinin ayrı auth sistemi kurması sürdürülemez	Kimlik Doğrulama & Secret Management
Sabit (hardcoded) iç çağrılar	Servis adresleri dinamik ölçeklenen ortamda sürekli değişir	Service Discovery & API Gateway
Tek bir log dosyasına bakmak	Bir isteğin hangi serviste, hangi adımda sorun yaşadığını bulmak zorlaşır	Observability (log/trace/metric)

Bu döküman, aşağıdaki bölümlerde tam olarak bu beş kategoriye tek tek ele alır.

3. Konu Başlıkları

3.1 Veri Katmanı (Data Layer)

3.1.1 PostgreSQL

Nedir? Açık kaynaklı, ilişkisel (relational) bir veritabanı yönetim sistemidir (RDBMS). Veriyi satır/sütun yapısındaki tablolarda tutar, SQL diliyle sorgulanır.

Temel kavramlar:

- **ACID:** Atomicity, Consistency, Isolation, Durability — bir transaction'ın ya tamamen ya da hiç uygulanmamasını garanti eden dört özellik.
- **Şema (Schema):** Tabloların, ilişkilerin ve kısıtların (constraint) tanımlandığı yapı.
- **Transaction:** Birden fazla işlemin “hep ya da hiç” mantığıyla tek bir birim olarak çalıştırılması.

Mikroservislerdeki rolü: Klasik “database per service” prensibinde her mikroservis kendi veritabanına sahip olur; güçlü tutarlılık gerektiren veriler (sipariş, ödeme, stok) genelde ilişkisel bir veritabanında tutulur.

3.1.2 Redis

Nedir? Bellek içinde (in-memory) çalışan bir key-value veri deposudur. Disk tabanlı veritabanlarına göre çok daha hızlıdır çünkü veriyi RAM'de tutar.

Temel kavramlar:

- **Cache-aside pattern:** Uygulama önce cache'e bakar; veri yoksa asıl veritabanından çekip cache'e yazar.
- **TTL (Time To Live):** Bir key'in ne kadar süre sonra otomatik silineceği.
- **Persistence (opsiyonel):** RDB (snapshot) veya AOF (append-only file) ile veriyi diske de yazabilir.
- **Distributed Lock:** Birden fazla servis instance'ının aynı kaynağa aynı anda erişmesini engellemek için kullanılan kilitleme mekanizması (örn. SETNX).

Mikroservislerdeki rolü: Cache, session store, rate limiting, distributed lock, pub/sub gibi birçok amaçla kullanılır.

3.2 Event-Driven Mimari ve Mesajlaşma Altyapısı

3.2.1 Apache Kafka

Nedir? Dağıtık, yüksek hacimli event/log akışlarını işlemek için tasarlanmış bir event streaming platformudur. Klasik bir mesaj kuyruğundan farklı olarak, mesajları sırayla bir “log” gibi diske yazar ve saklar.

Temel kavramlar:

- **Topic:** Event'lerin kategorize edildiği mantıksal kanal.
- **Partition:** Bir topic'in paralel işlenebilmesi için bölündüğü alt birimler; paralellik ve ölçeklenebilirlik buradan gelir.
- **Offset:** Bir partition içindeki her mesajın sıra numarası.
- **Consumer Group:** Aynı topic'i birlikte tüketen consumer'ların oluşturduğu grup; partition'lar grup içindeki consumer'lara dağıtılır.
- **Broker:** Kafka cluster'ındaki her bir sunucu/node.

- **Replication Factor:** Bir partition'ın kaç broker'da kopyalanacağı (tek broker'lı ortamda maksimum 1 olabilir).
- **KRaft:** Kafka'nın Zookeeper'a bağımlılığını ortadan kaldıran yeni mimarisi.

Neden kullanılır? Yüksek throughput gerektiren, event'lerin belirli bir süre saklanması gereken (event sourcing, audit log, stream processing) senaryolarda tercih edilir.

3.2.2 RabbitMQ

Nedir? Klasik anlamda bir mesaj kuyruğu (message broker)'dur. AMQP protokolünü kullanır.

Temel kavramlar:

- **Exchange:** Producer'ın mesajı gönderdiği, hangi kuyruğa yönlendireceğine karar veren bileşen.
- **Queue:** Mesajların consumer tarafından işlenene kadar biriktiği yapı.
- **Binding & Routing Key:** Exchange'in mesajı hangi kuyruğa yönlendireceğini belirleyen kurallar.
- **Ack/Nack:** Consumer'ın mesajı başarıyla işlediğini ya da işleyemediğini broker'a bildirmesi.
- **Durable Queue:** Broker yeniden başlasa bile kuyruğun kaybolmaması.

Kafka ile farkı:

Özellik	Kafka	RabbitMQ
Model	Log tabanlı, pull (consumer çeker)	Kuyruk tabanlı, push (broker iter)
Mesaj saklama	Belirli süre/boyut kadar saklanır (tüketilse de silinmez)	Tüketilip ack'lenince kuyruktan silinir
En uygun senaryo	Yüksek hacimli event stream, event sourcing	Klasik iş kuyruğu, RPC benzeri senaryolar
Ölçeklenebilirlik	Partition ile yatay ölçekleme	Cluster + queue mirroring

Bizim ortamımızdaki rolü: demo-queue üzerinden publish/consume senaryosu (manuel ack ile); RabbitMQ Management UI ile görsel takip.

3.3 Background Job Processing

3.3.1 Hangfire

Nedir? .NET uygulamaları için arka planda iş çalıştırmayı sağlayan bir kütüphanedir. Job'ların durumunu kalıcı bir storage'da (bizde PostgreSQL) saklar.

Temel kavramlar:

- **Fire-and-forget:** Hemen kuyruğa alınıp arka planda bir kez çalışan job.
- **Scheduled (delayed):** Belirli bir süre sonra çalışacak job.
- **Recurring:** Cron ifadesiyle tekrar tekrar çalışan job.
- **Dashboard:** Job'ların durumunu izlemek için gelen web arayüzü.

Mikroservislerdeki rolü: E-posta gönderimi, rapor oluşturma, periyodik veri temizliği gibi senkron istek içinde yapılmaması gereken işler için kullanılır.

3.4 Kimlik Doğrulama, Yetkilendirme ve Secret Management

3.4.1 HashiCorp Vault

Nedir? Hassas verilerin güvenli şekilde saklanması, erişiminin kontrol edilmesi ve dinamik olarak üretilmesi için kullanılan bir secret management aracıdır.

Temel kavramlar:

- **KV Secret Engine:** Basit key-value çiftlerinin saklandığı en yaygın secret engine.
- **Dynamic Secrets:** Vault'un, örneğin bir veritabanı için anlık, kısa ömürlü (TTL'li) kullanıcı adı/şifre üretmesi.
- **Token & Lease:** Vault'a erişim için kullanılan kimlik ve bu erişimin ne kadar süre geçerli olacağı.
- **Unseal:** Vault production modunda, sunucu yeniden başladığında şifreleme anahtarının parçalarıyla kilidinin açılması süreci.

Neden mikroservislerde kritik? Sırların kod içine yazılmaması, merkezi ve denetlenebilir şekilde yönetilmesi gerekir — “12 Factor App” prensiplerinden biridir.

3.4.2 Keycloak

Nedir? Açık kaynaklı bir Identity and Access Management (IAM) çözümüdür. OAuth2, OpenID Connect (OIDC) ve SAML protokollerini destekler.

Temel kavramlar:

- **Realm:** Kullanıcıların, rollerin ve client'ların izole edildiği en üst düzey yönetim birimi.
- **Client:** Keycloak'a kimlik doğrulama için bağlanan uygulama/servis tanımı.
- **Role:** Kullanıcıya atanan, hangi işlemleri yapabileceğini belirleyen yetki etiketi.
- **JWT:** Kimlik doğrulaması sonrası üretilen, imzalı ve kendi içinde bilgi taşıyan token.
- **SSO:** Bir kez giriş yapan kullanıcının aynı oturumla birden fazla uygulamaya erişebilmesi.

3.4.2.1 Authorization Grant Türleri (OAuth2 / OIDC Flows)

Keycloak, OAuth2 standardının tanımladığı birden fazla grant type (yetkilendirme akışı) destekler. Hangi grant'in kullanılacağı, “kim token istiyor” ve “client sırrını güvenli şekilde saklayabiliyor mu” sorularına göre belirlenir.

Grant Type	Nasıl Çalışır	Kullanım Senaryosu	Client Tipi
Authorization Code	Kullanıcı login sayfasına yönlendirilir, code ile geri döner; backend code'u token ile değiştirir	Klasik web uygulamaları (server-side)	Confidential
Authorization Code + PKCE	Aynısı, ek olarak code_verifier/code_challenge ile code interception'a karşı korunur	SPA, mobil uygulamalar	Public
Client Credentials	Kullanıcı yok; client kendi client_id/secret'ıyla doğrudan token alır	Servis-servis iletişimi	Confidential
Resource Owner Password Credentials	Kullanıcı adı/şifre doğrudan client'a girilip token alınır	Sadece güvenilir legacy senaryolar	Confidential
Refresh Token	Access token süresi dolunca, tekrar login olmadan yeni token alınır	Diğer flow'ların devamı	—
Device Authorization	Kısıtlı cihaz kod gösterir; kullanıcı başka cihazdan kodu girip yetkilendirir	Smart TV, CLI araçları	Public
Implicit (deprecated)	Token doğrudan URL fragment'ında dönerdi	Eskiden SPA'lar için	—

Güvenlik notu: OAuth 2.1 ile Implicit Grant tamamen kaldırılmıştır; SPA'lar için artık Authorization Code + PKCE standart öneridir. Resource Owner Password Credentials de güvenlik riskleri nedeniyle mümkün olduğunca kaçınılması gereken bir yöntemdir.

Bizim senaryomuzda hangisi mantıklı olurdu? Dışarıdan çağrılırken Authorization Code + PKCE, servisler arası iç çağrılarda Client Credentials grant'i tipik tercih olurdu.

3.4.2.2 .NET Core ile Keycloak Entegrasyonu

Keycloak standart bir OIDC/JWT sağlayıcısı olduğu için, .NET tarafında özel bir Keycloak SDK'sına gerek yoktur — Microsoft.AspNetCore.Authentication.JwtBearer paketi yeterlidir:

```
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.Authority = "http://localhost:8180/realms/neominal-realm";
        options.Audience = "neominal-app";           // Keycloak'taki client id
        options.RequireHttpsMetadata = false;         // SADECE dev ortamında
    });

builder.Services.AddAuthorization();
// ...
app.UseAuthentication();
app.UseAuthorization();
```

Sonrasında endpoint'ler .RequireAuthorization() (minimal API) veya [Authorize] attribute'u ile korunur. Tek dikkat edilmesi gereken nokta: Keycloak'un rollerini JWT içine realm_access.roles gibi iç içe (nested) bir claim olarak koyması — ASP.NET Core'un varsayılan rol mapping'i bunu doğrudan anlamaz, bu yüzden genelde küçük bir claims transformation veya custom authorization policy ile Keycloak rollerinin .NET'in ClaimTypes.Role'üne eşlenmesi gerekir. Tarayıcı tabanlı (cookie session'lı) klasik web uygulamaları için ise Microsoft.AspNetCore.Authentication.OpenIdConnect paketiyle tam bir Authorization Code akışı kurulabilir.

3.5 Service Discovery, Networking ve API Gateway

3.5.1 HashiCorp Consul

Nedir? Mikroservis ortamlarında service discovery, health checking ve merkezi key-value konfigürasyon deposu sağlayan bir araçtır.

Temel kavramlar:

- **Service Registration:** Bir servisin ayağa kalktığı anda kendini Consul'a kaydetmesi.
- **Service Discovery:** Bir servisin, ihtiyaç duyduğu başka bir servisin adresini Consul'a sorarak dinamik olarak bulması.
- **Health Check:** Consul'un kayıtlı servislere periyodik sağlık kontrolü yapması.
- **Consul Connect (Service Mesh):** Servisler arası trafiği şifreleyen gelişmiş özellik.

3.5.2 Nginx (Reverse Proxy)

Nedir? Yüksek performanslı bir web sunucusu ve reverse proxy'dir. Gelen istekleri arkasındaki uygulama sunucusuna yönlendirir.

Temel kavramlar:

- **Reverse Proxy:** İstemcinin doğrudan uygulama sunucusuyla değil, aradaki bir proxy ile konuşması.
- **Load Balancing:** Gelen trafiğin birden fazla instance arasında dağıtılması.
- **TLS Termination:** HTTPS şifrelemesinin proxy katmanında yapılması.
- **Forwarded Headers:** X-Forwarded-For, X-Forwarded-Proto ile gerçek istemci bilgisinin taşınması.

3.5.3 API Gateway: YARP ve Ocelot (.NET Ekosistemi)

Nginx genel amaçlı, dile bağımsız bir reverse proxy'dir. .NET ekosisteminde ise gateway mantığını C# kodu içinde yazmak/genişletmek isteyen ekipler için iki popüler alternatif vardır: YARP ve Ocelot.

YARP (Yet Another Reverse Proxy) nedir? Microsoft'un geliştirdiği, ASP.NET Core üzerine inşa edilmiş, kod olarak konfigüre edilebilen bir reverse proxy kütüphanesidir. Hazır bir gateway ürünü değil, kendi gateway'inizi inşa etmek için kullanılan bir framework/toolkit'tir.

Ocelot nedir? .NET için, YARP'tan önce en yaygın kullanılan, açık kaynak, “hepsi dahil” (batteries-included) bir API Gateway kütüphanesidir. Konfigürasyonu genelde JSON dosyası (ocelot.json) üzerinden yapılır.

Karşılaştırma:

Özellik	YARP	Ocelot
Geliştirici / Destek	Microsoft (resmi)	Topluluk (açık kaynak)
Performans	Çok yüksek — Kestrel native pipeline	İyi, biraz daha fazla overhead
Konfigürasyon	Kod-öncelikli + appsettings, esnek	Deklaratif JSON (ocelot.json)
Rate Limiting	Yok (native) — elle eklenir	Built-in
Response Caching	Yok (native)	Built-in (basit)
Request Aggregation	Yok — elle/custom yazılmalı	Built-in
Retry / Circuit Breaker	Elle Polly entegrasyonu gerekir	Built-in (Polly tabanlı QoS)
Service Discovery entegrasyonu	Yok (native)	Built-in (Consul, Eureka)
HTTP/2, HTTP/3 desteği	Native tam destek	Sınırlı

YARP'ın artıları: Microsoft'un resmi ürünü olması uzun vadeli güvence sağlar; performans ve modern protokol desteği çok daha güçlüdür; route/cluster/transform tamamen kod ile özelleştirilebilir.

YARP'ın eksileri: Hazır özellik seti dardır; daha çok bir kütüphane olduğu için ilk kurulumda daha fazla kod yazmayı gerektirir.

Ocelot'un artıları: Kurulumu hızlıdır; rate limiting, caching, aggregation, QoS built-in gelir; Consul/Eureka entegrasyonu hazır.

Ocelot'un eksileri: Performansı YARP'a göre biraz daha düşüktür; topluluk bakımı daha yavaş ilerleyebilir; çok özel senaryolarda YARP kadar esnek değildir.

Genel öneri: Basit, hızlı kurulan bir gateway için Ocelot; yüksek performans ve tam esneklik isteyen karmaşık senaryolar için YARP daha uygundur.

3.6 Gözlemlenebilirlik (Observability)

Observability, klasik “monitoring”den farklı olarak sistemin neden o durumda olduğunu anlamaya odaklanır ve üç temel sacayağına dayanır: Logs, Traces, Metrics.

3.6.1 Serilog + Seq — Loglar (Logs)

Serilog nedir? .NET için bir structured logging kütüphanesidir. Düz metin yerine her log kaydını key-value çiftleri olarak üretir.

Seq nedir? Serilog'un ürettiği logları toplayan, indeksleyen ve sorgulanabilir hale getiren bir log sunucusu/arayüzüdür.

Temel kavramlar: Enrichment (otomatik ek bilgi ekleme), Log Level (Verbose/Debug/Information/Warning/Error/Fatal), Sink (logların gönderildiği hedef).

3.6.2 OpenTelemetry + Jaeger — Dağıtık İzleme

Nedir? Bir isteğin birden fazla mikroservisten geçerken izlenmesini sağlayan sistemdir. OpenTelemetry veriyi üretir/toplar, Jaeger görselleştirir.

Temel kavramlar:

- **Trace:** Bir isteğin uçtan uca tuttuğu kayıt.
- **Span:** Bir trace içindeki tek bir işlem/adım.
- **Context Propagation:** Trace/span kimliğinin servisler arası taşınması.
- **OTLP:** Trace/metric verisinin toplayıcıya gönderildiği standart protokol.

3.6.3 Prometheus — Metrikler

Nedir? Zaman serisi tabanlı bir metrik toplama ve sorgulama sistemidir.

Mimari ve çalışma modeli

Prometheus'un en ayırt edici özelliği pull-based (çekme tabanlı) modelidir. Çoğu monitoring sisteminde (örn. StatsD, Datadog agent) uygulama metrikleri kendisi bir sunucuya gönderir (push). Prometheus'ta ise tam tersi olur: Prometheus, izlediği her hedefe (target) kendisi gider ve /metrics endpoint'ini periyodik olarak (varsayılan 15sn) HTTP GET ile "scrape" eder.

Bunun avantajları:

- Hedefin ayakta olup olmadığını (up/down) Prometheus doğrudan anlar — scrape başarısızsa up{job="..."} == 0 metriği otomatik oluşur.
- Uygulamanın "nereye push edeceğini" bilmesine gerek yok, sadece bir endpoint sunar.
- Test/debug kolaydır — curl http://localhost:8080/metrics ile Prometheus'un gördüğü ham veriyi siz de görebilirsiniz.

Dezavantajı: kısa ömürlü (ephemeral) job'lar (örn. bir cron job 3 saniyede biter) scrape edilmeden bitebilir — bunun için Prometheus'un Pushgateway eklentisi vardır (istisnai durum için push modeline geçiş).

Veri modeli

Her metrik, bir isim ve sıfır veya daha fazla label (key-value etiket) ile tanımlanır:

http_requests_total{method="POST", endpoint="/demo/db/products", status="201"} 42

Bu, PromQL'de son derece esnek sorgulamaya izin verir — örneğin sadece POST isteklerini, sadece belirli bir endpoint'i, ya da hepsini toplu filtreleyebilirsiniz. Ama dikkat: label cardinality patlaması (örneğin user_id'yi label yapmak) Prometheus'un belleğini/diskini çok hızlı şişirir — bu yaygın bir prod hatasıdır.

Metrik tipleri

Tip	Ne yapar	Örnek
Counter	Sadece artar (reset olabilir, ama asla azalmaz)	Toplam istek sayısı
Gauge	Anlık, artıp azalabilir	Aktif bağlantı sayısı, kuyruk uzunluğu
Histogram	Değerleri önceden tanımlı bucket'lara dağıtır, sunucu tarafında hesaplanır	İstek süresi dağılımı (<10ms, <50ms, <200ms...)
Summary	Histogram'a benzer ama percentile'ları (p50/p95/p99) client tarafında hesaplar	Aynı amaç, ama farklı topluluk (aggregation) matematiği

Histogram'lar server-side aggregate edilebilir (birden fazla instance'ın verisini toplayıp tek bir p95 hesaplayabilirsiniz), Summary'ler edilemez — bu yüzden mikroservis ortamlarında genelde Histogram tercih edilir.

PromQL örnekleri:

```
promql
# Son 5 dakikada saniyedeki istek oranı
rate(http_requests_total[5m])

# p95 istek süresi
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

# Hangi target'lar şu an DOWN
up == 0
```

Alertmanager: Prometheus'un kendisi sadece veri toplar/sorgular; alarm kurallarını (.rules dosyaları) değerlendirip bildirim (Slack, e-posta, PagerDuty) göndermek ayrı bir bileşen olan Alertmanager'ın işidir.

Sınırlılık: Prometheus'un yerel storage'ı tek node'ludur ve uzun süreli (aylar/yıllar) saklama için tasarlanmamıştır. Prod ortamlarda genelde Thanos, Cortex veya Grafana Mimir ile "uzun vadeli, çok node'lu" hale getirilir.

3.6.4 Grafana — Görselleştirme

Nedir? Prometheus, Loki, Jaeger gibi kaynaklardan gelen veriyi dashboard'lar hâlinde görselleştiren araçtır.

Grafana'yı Prometheus'a "bağımlı" bir araç sanmak yaygın bir yanlış — aslında kaynak-agnostik bir görselleştirme katmanıdır. Aynı dashboard içinde Prometheus'tan metrik, Loki'den log, Tempo/Jaeger'dan trace çekip yan yana gösterebilir.

Temel yapı taşları:

- **Data Source:** Grafana'nın sorgu göndereceği backend (Prometheus, MySQL, Elasticsearch, CloudWatch, Loki, Tempo — 100'den fazla resmi/topluluk data source'u var).

- **Dashboard:** Panel'lerin bir araya geldiği sayfa; JSON olarak export/import edilebilir (yani "dashboard as code" — Git'te versiyonlanabilir, Terraform/Grafonnet ile otomatik deploy edilebilir).
- **Panel:** Tek bir görsel (line chart, bar gauge, tablo, heatmap, stat kutusu vb.).
- **Variable (Templating):** Dashboard'un üstüne konan dropdown'lar (örn. "hangi servis?", "hangi environment?") — tek bir dashboard şablonu, seçime göre farklı veriyi gösterir.
- **Alerting:** Grafana'nın kendi alarm motoru da vardır (Alertmanager'dan bağımsız) — Grafana 8+ ile "Unified Alerting" adıyla, birden fazla data source'a dayalı alarm kuralları tek yerden yönetilebilir.
- **Explore:** Dashboard kurmadan, ad-hoc sorgu/log arama yapılan mod (Kibana'nın "Discover"ına benzer).

Grafana neden "yaygınlaştı"?

Kibana (Elasticsearch'e bağımlı) ve eski Datadog/New Relic gibi kapalı SaaS çözümlerin aksine, Grafana açık kaynak, self-hosted edilebilir ve her şeyi tek panelde birleştirme felsefesiyle (metrics+logs+traces = "observability üçlüsü") öne çıktı.

3.7 Management UI'lar

Aşağıdaki üç araç, altyapı bileşenlerinin kendi resmi web arayüzleridir — ayrı birer “ürün” değil, ilgili sistemin bir parçasıdır:

Araç	Hangi sistemin arayüzü	Ne işe yarar
Kafka UI	Apache Kafka	Topic, partition, mesaj ve consumer group'ları görsel olarak inceleme
RedisInsight	Redis	Key-value verilerini görsel olarak inceleme, sorgulama
RabbitMQ Management UI	RabbitMQ (aynı imajın içinde gelir)	Queue, exchange ve mesaj trafiğini görsel olarak izleme

Production ortamlarında bu tür araçlar genelde sadece iç ağdan erişilebilir tutulur, asla doğrudan internete açılmaz.

4. Kurulması Gereken Altyapılar (Checklist)

Aşağıdaki tüm servisler docker imajları üzerinden ayağa kaldırılır ve ilgili portlardan hizmet verirler.

#	Servis	İmaj	Host Portu	Kategori
1	PostgreSQL	postgres:16-alpine	15432	Veri Katmanı
2	Redis	redis:7-alpine	6379	Veri Katmanı
3	Kafka (KRaft)	apache/kafka:latest	19094	Mesajlaşma
4	Kafka UI	ghcr.io/kafbat/kafka-ui:latest	8082	Geliştirici Aracı
5	RabbitMQ (+ UI)	rabbitmq:3-management-alpine	25672 / 15672	Mesajlaşma
6	HashiCorp Vault	hashicorp/vault:1.17	8200	Secret Management
7	Keycloak	quay.io/keycloak/keycloak:21.1.1	8180	Identity & Access Mgmt
8	HashiCorp Consul	hashicorp/consul:1.19	8500	Service Discovery
9	Seq	datalust/seq:latest	8081 / 5341	Observability - Logs

#	Servis	İmaj	Host Portu	Kategori
10	Jaeger	jaegertracing/all-in-one:1.57	16686 / 4317	Observability - Tracing
11	Prometheus	prom/prometheus:latest	9090	Observability - Metrics
12	Grafana	grafana/grafana:latest	3000	Observability - Visualization
13	RedisInsight	redis/redisinsight:latest	5540	Geliştirici Aracı
14	Nginx	nginx:alpine	8090	Reverse Proxy

Not: Bazı servislerin host portları (Postgres: 15432, Kafka: 19094, RabbitMQ AMQP: 25672) standart değerlerinden kasıtlı olarak kaydırılmıştır — bazı Windows güvenlik yazılımlarının bilinen veritabanı/mesajlaşma protokol portlarını izleyip host/container bağlantısını kesebilmesi riskine karşı alınmış önleyici bir tedbirdir.

Donanım notu: 14 servisin aynı anda çalışması için en az 8 GB boş RAM ve 20 GB disk alanı önerilir.

5. Özet Karşılaştırma Tablosu

Bileşen	Kategori	Temel Amaç	Bilinen Alternatifleri
PostgreSQL	Veritabanı	İlişkisel, ACID uyumlu veri saklama	MySQL, SQL Server, Oracle
Redis	Cache / In-memory store	Hızlı erişim, cache, lock, pub/sub	Memcached, Hazelcast
Kafka	Event Streaming	Yüksek hacimli, kalıcı event akışı	Apache Pulsar, AWS Kinesis
RabbitMQ	Message Queue	Klasik iş kuyruğu, esnek routing	ActiveMQ, Amazon SQS
Hangfire	Background Jobs	.NET içi zamanlanmış/gecikmeli işler	Quartz.NET, Azure Functions
Vault	Secret Management	Merkezi, dinamik secret yönetimi	AWS Secrets Manager, Azure Key Vault
Keycloak	Identity & Access Mgmt	Merkezi kimlik doğrulama (OIDC/SAML)	Auth0, Okta, Azure AD
Consul	Service Discovery	Dinamik servis bulma, health check	Eureka, K8s Service Discovery
Nginx	Reverse Proxy	Tek giriş noktası, load balancing	HAProxy, Traefik, YARP
YARP	.NET API Gateway	Yüksek performans, özelleştirilebilir proxy	Ocelot, Nginx, Envoy
Ocelot	.NET API Gateway	Hazır özellikli gateway	YARP, Kong, Spring Cloud Gateway
Serilog + Seq	Logging	Structured log toplama/sorgulama	ELK Stack
OpenTelemetry + Jaeger	Distributed Tracing	Uçtan uca istek izleme	Zipkin, AWS X-Ray
Prometheus	Metrics	Zaman serisi metrik toplama	InfluxDB, Datadog
Grafana	Visualization	Dashboard ve alerting	Kibana, Datadog Dashboards

6. Bu Bileşenler Birlikte Nasıl Çalışır?

- İstemci isteği Nginx'e (ya da .NET-native bir gateway kullanılıyorsa YARP/Ocelot'a) gelir → gateway isteği .NET uygulamasına yönlendirir.
- Uygulama, iş mantığına göre PostgreSQL'den veri okur/yazar, Redis'te cache kontrolü yapar.

3. Bir event yayınlanması gerekiyorsa Kafka veya RabbitMQ'ya mesaj gönderilir; arka planda çalışan consumer'lar bunları işler.
 4. Zamanlanmış/gecikmeli bir iş varsa Hangfire devreye girer.
 5. Hassas bir bilgiye ihtiyaç varsa Vault'tan runtime'da çekilir.
 6. Kullanıcının kimliği doğrulanması gerekiyorsa Keycloak üzerinden JWT alınır/doğrulur.
 7. Uygulama başka bir servise ihtiyaç duyduğunda servis adresini Consul'dan sorar.
 8. Bu süreç boyunca her adım Serilog'la loglanır (Seq'te görünür), OpenTelemetry ile izlenir (Jaeger'da görünür), performans Prometheus tarafından ölçülür (Grafana'da görselleşir).
- Bu, gerçek bir production mikroservis mimarisinin küçük ölçekli ama kavramsal olarak bir anlatımıdır.

7. Katılımcı İçin Ön Çalışma Önerileri

- Docker ve Docker Compose temel komutlarına (up, down, ps, logs) aşina olunması.
- REST API / HTTP protokolü temel kavramlarının (status code, header, method) tazelenmesi.
- OAuth2/OIDC'nin ne problemi çözdüğüne dair genel bir fikir edinilmesi.
- “12 Factor App” prensiplerinin (özellikle Config ve Backing Services maddeleri) bir kez okunması.
- Daha önce hiç mesaj kuyruğu (Kafka/RabbitMQ) ile çalışılmadıysa, publish/subscribe ve producer/consumer terimlerinin temel düzeyde bilinmesi.